Prérequis fondamentaux et structure d'un projet de Machine Learning

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Prérequis mathématiques et statistiques **3**

A. Introduction 3

B. Fondamentaux mathématiques 3

C. Fondamentaux statistiques 11

II - Librairies fondamentales en Python **12**

III - Cycle de vie d'un projet de Machine Learning **20**

IV - Préparation et nettoyage des données **22**

V - Structuration d'un projet de Machine Learning **23**

VI - L'essentiel **24**

VII - Synthèse **25**

VIII - Auto-évaluation **28**

A. Test 28

B. Exercice 31

Solutions des exercices **32**

Tous droits réservés à STUDI - Reproduction interdite

I Prérequis mathématiques et statistiques

A. Introduction

L'intelligence artificielle et la data science sont devenues incontournables dans notre quotidien, des recommandations de films/séries aux voitures autonomes. Se cachent derrière ces technologies des principes fondamentaux souvent mathématiques et statistiques. Nous essaierons de comprendre comment ces concepts nous aident à donner du sens aux montagnes de données qui nous entourent.

Si nous imaginons les données comme un immense puzzle, les mathématiques nous fournissent les règles du jeu pour assembler les différentes pièces, tandis que les statistiques nous aident à comprendre l'image globale qui se dessine et les relations des pièces entre elles. Si nous prenons le cas de l'algèbre linaire, une branche des mathématiques, celle-ci nous permet de manipuler de grandes quantités d'informations de manière organisée, comme si nous rangions des milliers de photos dans des albums bien précis. Le calcul différentiel, autre notion importante des mathématiques, est comme une boussole pour l'intelligence artificielle. Il aide les algorithmes à trouver le meilleur chemin pour atteindre un objectif, lui permettant ainsi de s'améliorer en continu. Les probabilités nous aident à gérer l'incertitude. Elles donnent des outils pour évaluer les chances qu'un événement se produise. En data science, on prédit la probabilité qu'un client achète un produit ou qu'un système tombe en panne : c'est une manière de quantifier le risque et d'anticiper l'avenir. Les statistiques nous offrent des méthodes pour résumer et interpréter les données, elles nous permettent de détecter des tendances. Nous pouvons transformer des chiffres bruts en graphiques parlants, rendant les informations accessibles à tous.

Grâce à l'informatique moderne, des milliers de librairies sont accessibles ; nous les utiliserons pour être plus autonome et naviguer dans le monde de l'intelligence artificielle et de la data science sans être un mathématicien chevronné.

B. Fondamentaux mathématiques

Les vecteurs et matrices peuvent être considérés comme des **conteneurs de données**. Imaginons un vecteur comme une simple liste de chiffres, chaque chiffre peut représenter une caractéristique d'un objet ou d'une personne. Si nous prenons comme exemple un étudiant, un vecteur pourrait être : [âge, taille, note moyenne]. C'est une manière compacte de stocker des informations liées.

Exemple de vecteur — Exemple

Étudiant : [20,175, 14.5]

Une matrice est un tableau de chiffres, avec des lignes et des colonnes. En data science, elle est souvent utilisée pour représenter un **ensemble de données**. Chaque ligne peut être une observation ou une information, dans notre cas un étudiant, et chaque colonne une caractéristique (âge, taille, note). Ainsi, nous avons une nouvelle manière structurée de présenter l'information.

Exemple de matrice — Exemple

Classe :

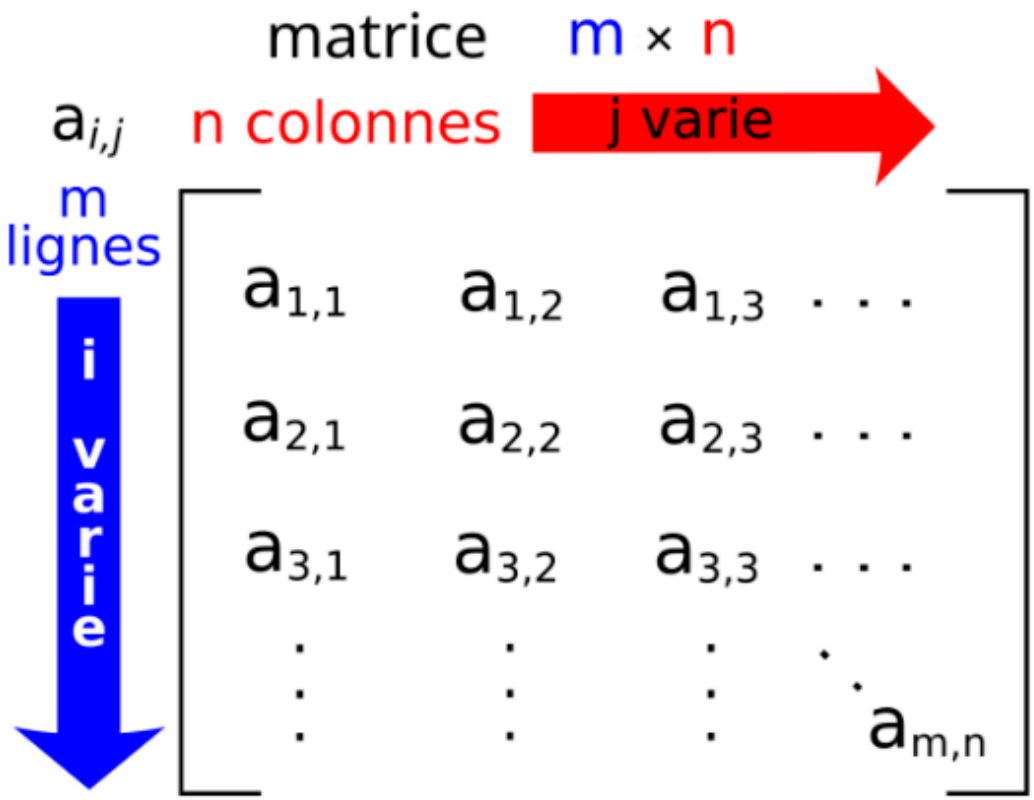
[20, 175, 14.5]

[21, 180, 16.0]

[19, 170, 13.0]

Tous droits réservés à STUDI - Reproduction interdite

Schéma d'une matrice



Pour multiplier 2 matrices, on multiplie les éléments des lignes de la première matrice par les éléments des colonnes de la seconde matrice, puis on additionne les résultats. Cela ressemble à un produit en croix étendu.

Exemple simple de multiplication matricielle Exemple

Matrice A = [1, 2] Matrice B = [5, 6]

[3, 4] [7, 8]

Résultat C = A x B

C[1,1] = (1 * 5) + (2 * 7) = 5 + 14 = 19

C[1,2] = (1 *6) + (2 * 8) = 6 + 16 = 22

C[2,1] = (3 * 5) + (4 * 7) = 15 + l28 = 43

C[2,2] = (3 * 6) + (4 * 8) = 18 + 32 = 50

Matrice C = [19, 22]

[43, 50]

Nous venons de voir comment résoudre un calcul simple, mais quelles sont les applications de l'algèbre linéaire en intelligence artificielle et data science ?

Si nous prenons le cas des réseaux de neurones, concept que nous approfondirons dans la suite du cours, quand celui-ci « *apprend* », il effectue en réalité énormément de multiplications matricielles pour ajuster ses connexions et améliorer ses prédictions.

Tous droits réservés à STUDI - Reproduction interdite

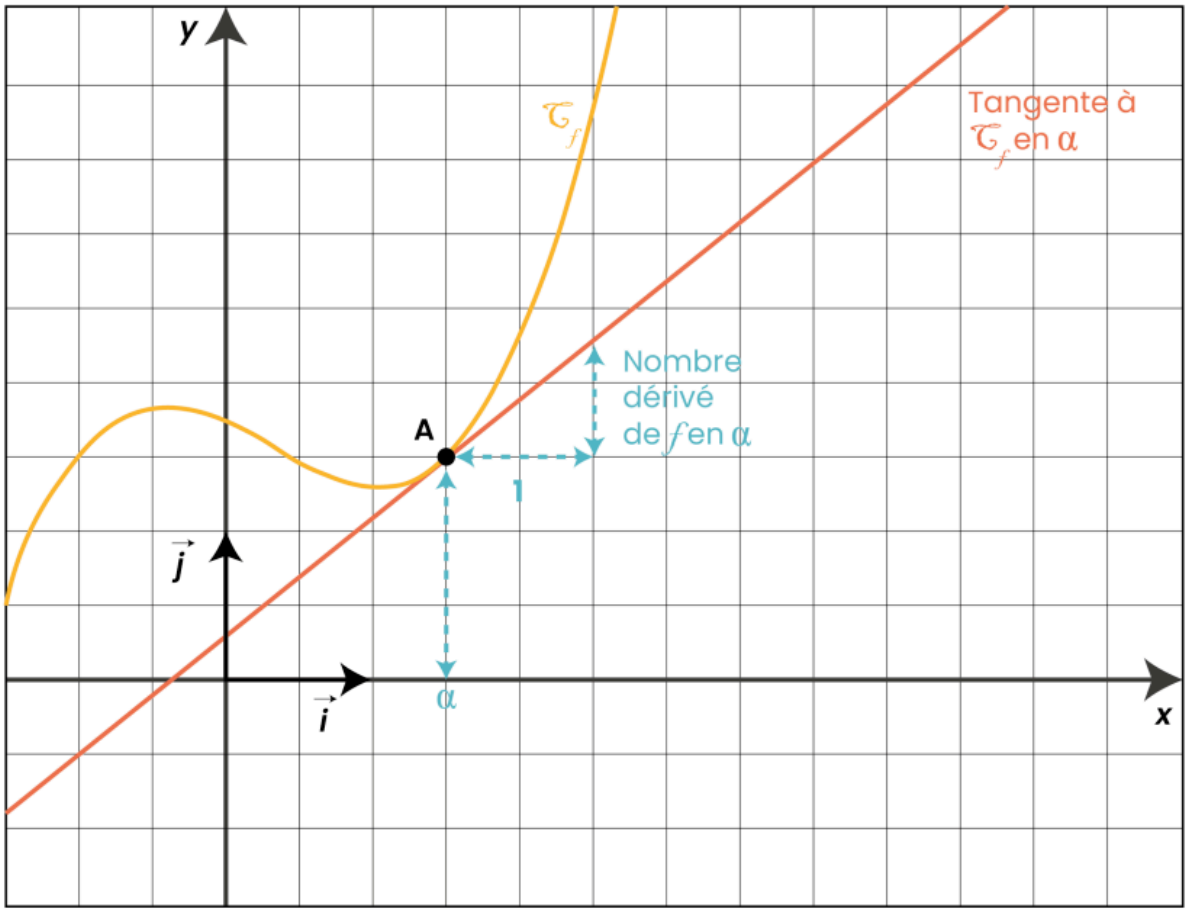
Dans le cas de la réduction de dimensionnalité, c'est aussi une opération utile, car parfois les données ont trop de caractéristiques (dimensions), ce qui les rend difficiles à analyser. L'algèbre linéaire permet de réduire ce nombre de dimensions tout en gardant l'information essentielle, comme sur une carte très détaillée, en la simplifiant et en ne gardant que les routes principales.

Dans un système de recommandation, Netflix suggère un film ou Amazon un produit par l'analyse de vos préférences et de celles d'autres utilisateurs qui disposent de similarités sous forme de matrices.

Abordons les notions de **dérivées** et **gradients** qui permettent de mesurer un changement sur une courbe.

Imaginons que nous marchons sur une colline ; la dérivée indique la pente de la colline à un endroit précis. Si la pente ascendante est forte, la dérivée est grande, si la pente de la colline se fait de plus en plus plate, la dérivée est proche de zéro. En mathématiques, la dérivée d'une fonction nous indique quand celle-ci change, ainsi que ses entrées.

Schéma d'une dérivée



Intéressons-nous à un concept important sans rentrer dans les détails : le gradient ou **vecteur gradient**. Le gradient d'une fonction multivariable comme f(x,y) est un vecteur qui indique la direction de la plus forte augmentation de la fonction et dont la norme (longueur) indique à quel point la valeur de la fonction augmente rapidement dans cette direction.

Si nous prenons à nouveau notre exemple de colline, et que la hauteur de la colline est donnée par une fonction f(x,y), le gradient nous montre dans quelle direction monter pour grimper plus vite.

Tous droits réservés à STUDI - Reproduction interdite

La formule du gradient

Az Définition

Le gradient est une généralisation de la dérivée pour les fonctions qui dépendent de plusieurs variables. Le gradient indique la direction la plus raide.

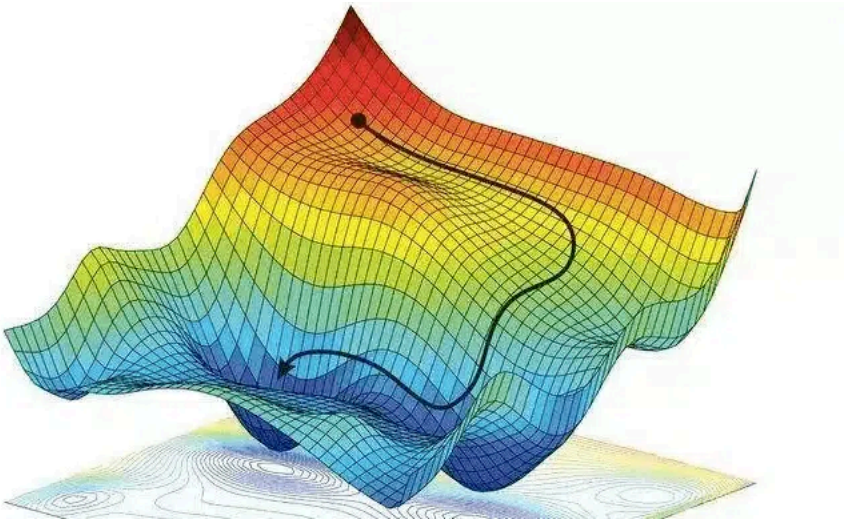
$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

Nous avons parlé du cas de la montée de notre colline ; qu'en est-il si nous souhaitons à l'inverse descendre ? La descente de gradient est un algorithme clé en intelligence artificielle. Mais cette fois-ci, nous sommes au sommet de la colline sans forcément avoir une vision claire sur le dénivelé et nous souhaitons atteindre le point le plus bas. Dans ce cas, nous ferons un pas dans la direction où la pente est la plus basse, puis un autre selon la même logique, et ainsi de suite jusqu'à atteindre le fond de la vallée.

Tous droits réservés à STUDI - Reproduction interdite

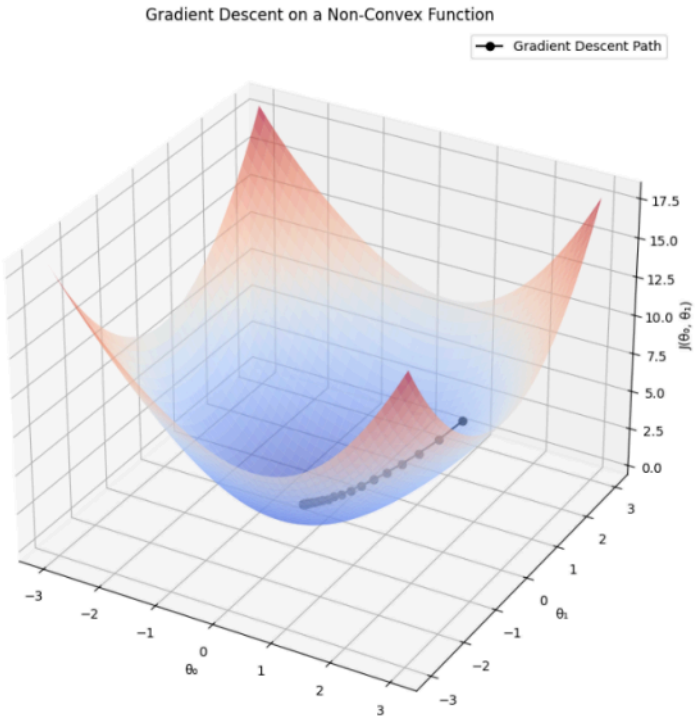
La descente de gradient

Cette image a été réalisée à partir d'un prompt simple et d'une illustration de gradient descent.



Tous droits réservés à STUDI - Reproduction interdite

Courbe créée avec matplotlib



Cette image peut être créée grâce à la librairie matplotlib avec le code Python suivant :

Code python

```python
# DEBUT DU CODE
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Fonction non convexe
def cost_function(theta0, theta1):
    return np.sin(theta0) * np.cos(theta1) + theta0**2 + theta1**2

# Gradient de la fonction
def gradient(theta0, theta1):
    dtheta0 = np.cos(theta0) * np.cos(theta1) + 2 * theta0
    dtheta1 = -np.sin(theta0) * np.sin(theta1) + 2 * theta1
    return dtheta0, dtheta1

# Initialisation
theta0, theta1 = 1.5, 1.5
learning_rate = 0.1
iterations = 50

# Historique
theta0_history = [theta0]
theta1_history = [theta1]
J_history = [cost_function(theta0, theta1)]

# Descente de gradient
for _ in range(iterations):
    dtheta0, dtheta1 = gradient(theta0, theta1)
    theta0 -= learning_rate * dtheta0
    theta1 -= learning_rate * dtheta1
    theta0_history.append(theta0)
    theta1_history.append(theta1)
    J_history.append(cost_function(theta0, theta1))

```


Tous droits réservés à STUDI - Reproduction interdite

```
35 # Surface de coût
36 theta0_vals = np.linspace(-3, 3, 100)
37 theta1_vals = np.linspace(-3, 3, 100)
38 T0, T1 = np.meshgrid(theta0_vals, theta1_vals)
39 J = cost_function(T0, T1)
40
41 # Affichage 3D
42 fig = plt.figure(figsize=(12, 8))
43 ax = fig.add_subplot(111, projection='3d')
44 ax.plot_surface(T0, T1, J, cmap='coolwarm', alpha=0.6)
45
46 # Trajectoire de la descente
47 ax.plot(theta0_history, theta1_history, J_history, color='black', marker='o',
   label='Gradient Descent Path')
48
49 ax.set_title('Gradient Descent on a Non-Convex Function')
50 ax.set_xlabel('θ₀')
51 ax.set_ylabel('θ₁')
52 ax.set_zlabel('J(θ₀, θ₁)')
53 ax.legend()
54 plt.tight_layout()
55 plt.show()
56 # FIN DU CODE
```

Il suffit de copier-coller sur cet outil en ligne : python-fiddle[1]

En apprentissage automatique, cette méthode est utilisée pour minimiser l'erreur d'un modèle. La formule de base de la descente est simple à comprendre.

La formule du gradient descent — Az Définition

$$x_{nouveau} = x_{ancien} - \alpha \cdot \nabla f\left(x_{ancien}\right)$$

Nouveaux paramètres = Anciens paramètres - Taux d'apprentissage × Gradient de l'erreur

Le taux d'apprentissage est la taille du pas que nous faisons à chaque itération : s'il est trop grand, nous risquons de rater le minimum. En revanche, s'il est trop petit, l'apprentissage sera très lent.

Avant de passer aux fondamentaux statistiques, abordons les **probabilités**. Celles-ci nous donnent un cadre pour comprendre et gérer l'incertitude. Ce qui est important à la fois en intelligence artificielle et en data science, c'est le fait que le monde réel est rarement noir ou blanc, et les probabilités nous aident à quantifier les chances qu'un évènement se produise.

La probabilité d'un événement est simplement une mesure de la chance qu'il se produise. Elle est toujours comprise entre 0 (impossible) et 1 (certain).

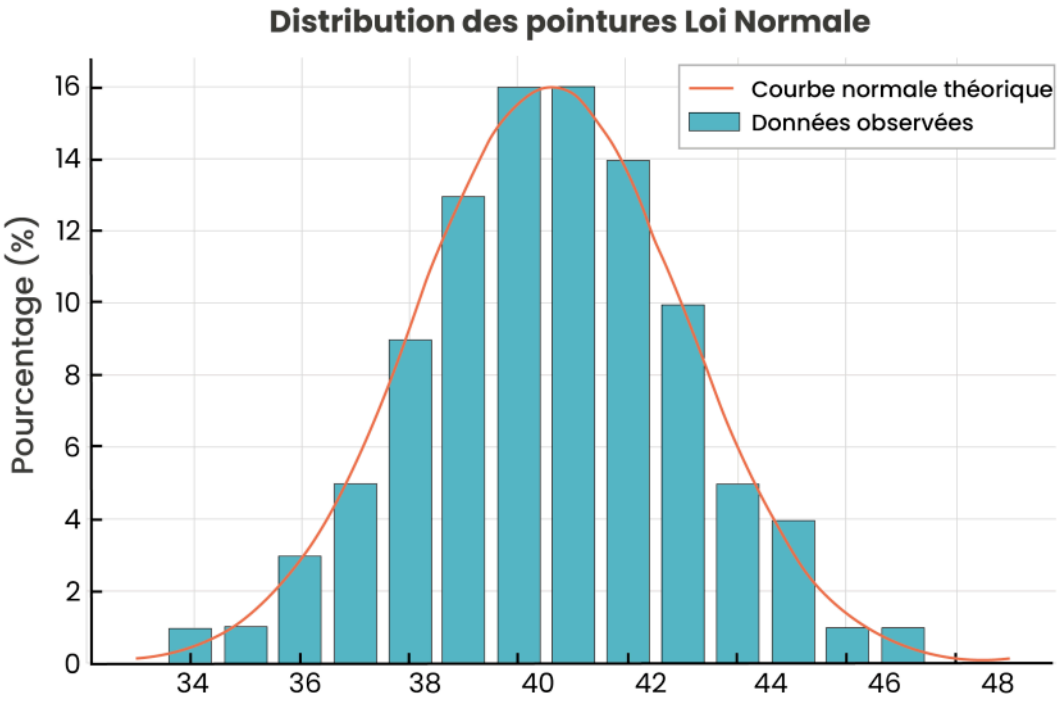
Prenons un lancer de pièce : la probabilité d'obtenir « *face* » en lançant la pièce est de 0,5 (ou 50 %). Une variable aléatoire est une quantité dont la valeur dépend du résultat d'un événement aléatoire. Par exemple, le nombre de fois où vous obtenez « *face* » en lançant une pièce 10 fois est une variable aléatoire.

Une distribution de probabilité nous montre comment les différentes valeurs d'une variable aléatoire sont réparties. La plus célèbre est la distribution normale, souvent appelée « *courbe en cloche* ». Beaucoup de phénomènes naturels, comme la taille des êtres vivants selon une espèce, suivent cette distribution.

1. https://python-fiddle.com/

Tous droits réservés à STUDI - Reproduction interdite

Schéma de la Loi normale



Ici le schéma montre un exemple de distribution des pointures d'une boutique en ligne. Les barres bleues représentent les données réelles issues des ventes d'une boutique en ligne. La courbe rouge est une loi normale (courbe en cloche) ajustée aux données observées.

Sans rentrer dans les détails, notons que la plupart des valeurs sont proches de la moyenne, qui ici serait une pointure en 40,5. Et que plus l'on s'éloigne de cette moyenne, moins les ventes dans ces pointures sont fréquentes. Cette courbe est symétrique : autant de valeurs au-dessus que de valeurs en dessous de la moyenne.

Nous pouvons interpréter ces données par une lecture visuelle de la courbe, et faire quelques affirmations :

- La majorité des clients achètent des pointures proches du 40-42,
- Environ 68 % des ventes se font dans l'intervalle [38 ; 43],
- Les ventes dans des pointures extrêmes sont très rares, ici ce sont des pointures en 34 et 48.

Que se passe-t-il si nous décidons d'ajouter à notre inventaire une nouvelle pointure en vente ? Est-ce que la courbe changerait ?

Parlons du théorème de Bayes, qui va permettre de mettre à jour la probabilité d'un événement lorsque nous obtenons de nouvelles informations.

La formule du théorème de Bayes

Az Définition

P (A | B) = P (B | A) * P (A) / P (B)

P (A | B) est la probabilité de B sachant A.

P (A) est la probabilité de A (avant de connaître B).

P (B) est la probabilité de B.

Tous droits réservés à STUDI - Reproduction interdite

C. Fondamentaux statistiques

Les **statistiques descriptives** sont comme un résumé de vos données, elles vous aident à comprendre les caractéristiques principales d'un ensemble d'informations sans avoir à regarder chaque donnée individuellement. C'est la première étape pour donner du sens à un grand volume de chiffres et données.

Reprenons certains concepts de base :

- **La moyenne**

  Elle correspond à la somme de toutes les valeurs divisées par le nombre de valeurs. Moyenne = (somme de toutes les valeurs) / nombre de valeurs

- **La médiane**

  C'est la valeur du milieu quand toutes les données sont rangées par ordre. Si nous avons un nombre pair de données, c'est la moyenne des 2 valeurs du milieu. Elle est moins sensible aux valeurs extrêmes que la moyenne.

- **Le mode**

  C'est la valeur qui apparaît le plus souvent dans notre ensemble de données.

Une fois que nous avons des données à disposition, nous pouvons étudier les mesures de dispersion. Pour savoir comment nos données sont étalées, dispersées ou regroupées :

- **L'étendue**

  C'est la différence entre la valeur la plus grande et la plus petite. C'est une mesure simple, mais qui peut être très influencée par les valeurs extrêmes.

- **L'écart-type**

  C'est la mesure la plus utilisée pour la dispersion. Un petit écart-type signifie que les données sont proches de la moyenne, un grand écart-type signifie qu'elles sont très étalées. C'est aussi la racine carrée de la variance : écart-type = racine carrée de la variance.

- **La variance**

  Elle mesure la dispersion des données autour de la moyenne. Plus la variance est grande, plus les données sont éloignées de la moyenne.

Tous droits réservés à STUDI - Reproduction interdite

II Librairies fondamentales en Python

Nous allons explorer les librairies Python les plus fondamentales pour la data science, telles que : NumPy, Pandas, Matplotlib, Seaborn et Scikit-learn. Chacune de ces librairies joue un rôle important à différentes étapes du processus d'analyse de données et de la préparation à la modélisation.

NumPy

NumPy (Numerical Python) est la librairie fondamentale pour le calcul scientifique en Python. Elle fournit de puissantes méthodes pour manipuler des tableaux multidimensionnels et des outils pour effectuer des opérations mathématiques et logiques sur ces tableaux. NumPy sert aussi de base pour d'autres librairies de la data science.

Exemples d'utilisation basique de la librairie NumPy

Méthode

Pour commencer, importons la librairie NumPy dans notre espace Python :

```
1 import numpy as np
```

Créons un tableau à partir d'une liste Python :

```
1 arr = np.array([1,2,3,4,5])
2 print(arr)
3
4 # Resultat : [1,2,3,4,5]
```

Créons un tableau multidimensionnel :

```
1 arr_2d = np.array([[1,2,3],[4,5,6]])
2 print(arr_2d)
3
4 # Resultat :
5 # [[1 2 3 ]
6 # [ 4 5 6 ]]
```

Des attributs/propriétés sont disponibles pour connaître les dimensions et formes d'un tableau :

- `ndim` renvoie le nombre de dimensions (axes) du tableau,
- `shape` renvoie un tuple qui indique la taille du tableau dans chaque dimension.

```
1 print(arr_2d.ndim)
2 # Résultat : 2
3
4 print(arr_2d.shape)
5 # Résultat : (2,3)
```

L'un des grands avantages de NumPy est sa capacité à effectuer des opérations arithmétiques sur des tableaux entiers sans avoir besoin d'utiliser de boucles. C'est ce que l'on appelle le calcul vectorisé. Prenons quelques exemples d'opérations arithmétiques.

```
1 arr1 = np.array([1, 2, 3])
2 arr2 = np.array([4, 5, 6])
3
4 # Addition
5 print(arr1 + arr2)
6 # Résultat : [5 7 9]
7
8 # Multiplication
9 print(arr1 * 2)
10 # Résultat : [2 4 6]
```


Tous droits réservés à STUDI - Reproduction interdite

NumPy dispose également de fonctions universelles qui s'appliquant à chaque élément d'un tableau :

```
# Racine carrée de chaque élément
print(np.sqrt(arr1))
# Résultat: [1. 1.41421356 1.73205081]

# Exponentielle de chaque élément
print(np.exp(arr1))
# Résultat: [ 2.71828183 7.3890561 20.08553692]
```

En résumé, NumPy est une librairie importante pour tout calcul numérique en Python. Connaître ses concepts est une base et une étape incontournable dans le monde de la data science.

Pandas

Pandas est une libraire tout aussi incontournable pour la manipulation de données. Construite sur NumPy, elle offre des structures de données flexibles et performantes, notamment à travers les Series et DataFrames, qui sont idéales pour travailler avec des données tabulaires comme des feuilles de calcul ou des bases de données.

Une *Series* est un tableau unidimensionnel (une colonne) capable de contenir n'importe quel type de données (nombres, chaînes de caractères, objets). Elle possède un index qui permet d'accéder aux éléments.

Le *DataFrame* est une structure de données tabulaire à 2 dimensions (lignes et colonnes), similaire à une feuille de calcul Excel, ou une autre table de base de données. C'est la structure la plus utilisée en data science avec Pandas.

Exemples d'utilisation basique de la librairie Pandas

Méthode

Voici comment importer la libraire Pandas :

```
import pandas as pd
```

Pandas facilite grandement l'importation de données depuis divers formats. Les plus courants sont les fichiers CSV (*Comma Separated Values*) et les fichiers Excel.

Voici comment charger ces fichiers :

```
# Charger un fichier CSV
df_csv = pd.read_csv('mon_fichier.csv')

# Charger un fichier Excel
df_excel = pd.read_excel('mon_fichier.xlsx')
```

Voici quelques exemples sur les structures de base : les Series.

Il s'agit d'un tableau avec un index, très semblable à un tableau de NumPy, mais avec étiquettes.

```
# Exemple de code


import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)

# Résultat :

a    10
b    20
```

Tous droits réservés à STUDI - Reproduction interdite

```
c    30
dtype: int64
```

Le DataFrame est quant à lui un tableau 2 dimensions (lignes et colonnes), équivalent à une feuille Excel.

Voici un exemple :

```
data = {
    'Nom': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Ville': ['Paris', 'Lyon', 'Marseille']
}

df = pd.DataFrame(data)
print(df)

# Résultat :

      Nom  Age      Ville
0   Alice   25      Paris
1     Bob   30       Lyon
2  Charlie   35  Marseille
```

Si nous souhaitons faire une inspection rapide des données, voici quelques attributs importants à connaître :

```
df.head()       # premières lignes
df.tail()       # dernières lignes
df.info()       # infos sur les colonnes et types
df.describe()   # statistiques descriptives
df.shape        # (nb_lignes, nb_colonnes)
df.columns      # noms des colonnes
df.index        # index des lignes
```

Parfois, nous pouvons avoir certains datasets avec des valeurs manquantes ou des données que nous avons besoin de transformer en valeurs numériques. Par exemple, si une colonne se réfère au genre d'une personne, en comptant la valeur « *male* » ou « *female* », nous pouvons supprimer/remplacer ces valeurs soit par un 0 ou 1. Ce qui sera plus facile plus tard pour interpréter nos données.

`drop()` sert à supprimer des lignes ou colonnes.

Voici un exemple :

```
df.drop('Ville', axis=1)    # supprime colonne
df.drop(0, axis=0)          # supprime ligne
```

`fillna()` sert à remplacer des valeurs manquantes.

`df.fillna(0)` va remplacer les valeurs manquantes par 0, souvent nous l'utilisons pour que notre algorithme ne gère pas les NaN (NotANumber).

`df.dropna()` supprime les lignes/colonnes avec des valeurs manquantes.

`rename()` est très utile si nous souhaitons par exemple transformer des valeurs pour des questions de présentation ou compréhension, voire aussi de formatage.

`df.rename(columns={'Âge': 'Age'})` permet de renommer des colonnes ou index.

Matplotlib et Seaborn sont 2 librairies très populaires pour créer des visualisations statiques. Elles sont extrêmement importantes dans la visualisation de données et l'exploration de données. Elles permettent de comprendre, repérer les tendances, des motifs, des valeurs ou des relations entre variables, que ne pourraient révéler de simples chiffres. Ces librairies servent aussi à présenter des analyses complexes de manière intuitive, ce qui est très pratique pour un public non technique. Les 2 librairies offrent de nombreuses options pour personnaliser vos graphiques.

Tous droits réservés à STUDI - Reproduction interdite

Avant de pouvoir utiliser Matplotlib, il est important de bien l'avoir installé sur sa machine. Si vous avez installé Python sur votre machine et pip (Python's package installer) , voici la commande :

```
pip install matplotlib
```

Si votre installation s'est bien déroulée, vous devriez avoir un écran similaire depuis votre terminal.

Installation de la librairie Matplotlib

Messages du terminal après commande :

```
ismae@iamniters MINGW64 /c/Python  Studi/PythonLab
$ pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (3.8.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (from
 matplotlib) (1.1.1)
Requirement already satisfied: cycler>=0.10 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (from mat
plotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (fro
m matplotlib) (4.43.1)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (fro
m matplotlib) (1.4.5)
Requirement already satisfied: numpy<2,>=1.21 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (from m
atplotlib) (1.26.1)
Requirement already satisfied: packaging>=20.0 in c:\users\ismae\appdata\roaming\python\python312\site-packages (from matplotlib)
 (23.2)
Requirement already satisfied: pillow>=6.2.0 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (from ma
tplotlib) (10.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\ismae\appdata\local\programs\python\python312\lib\site-packages (from
 matplotlib) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\ismae\appdata\roaming\python\python312\site-packages (from matplo
tlib) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\users\ismae\appdata\roaming\python\python312\site-packages (from python-dateutil>=2
.7->matplotlib) (1.16.0)

[notice] A new release of pip is available: 23.2.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Nous pouvons maintenant importer depuis notre fichier matplotlib de la manière suivante :

```
import matplotlib.pyplot as plt
```

Essayons de tracer une droite simple avec la séquence de code suivante :

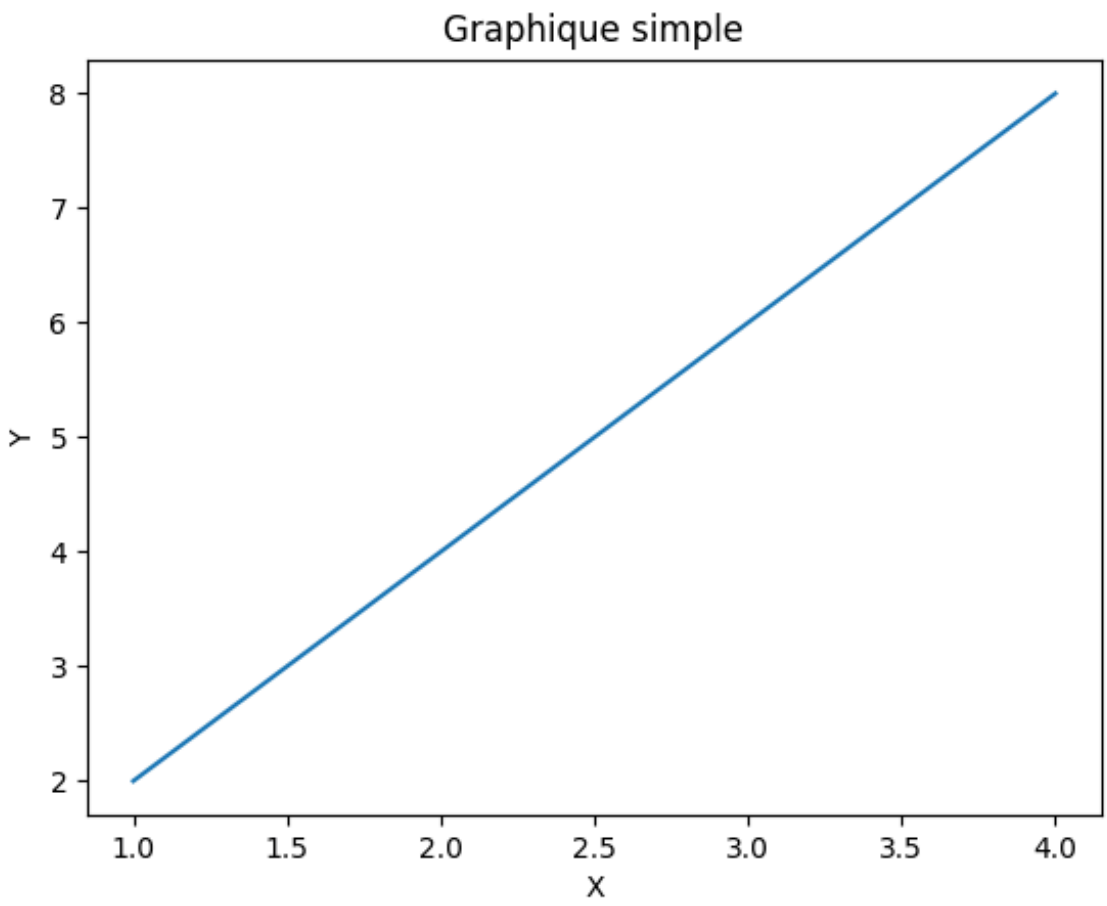
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.plot(x, y)            # tracer une ligne
plt.title("Graphique simple")
plt.xlabel("X")
plt.ylabel("Y")
.show()
```

Important

Il est essentiel d'avoir `plt.show()` pour afficher le graphique dans un script. Si vous utilisez votre IDE habituel, il est nécessaire d'utiliser la commande python `'nom_de_votre_fichier_python'`

Tous droits réservés à STUDI - Reproduction interdite

Voici ce que doit afficher votre ordinateur :



Voici quelques méthodes importantes :

`plot(x, y)` Tracer une courbe

`scatter(x, y)` Nuage de points

`bar(x, height)` Histogramme en barres

`hist(data)` Histogramme

`title(str)` Titre du graphique

`xlabel(str) / ylabel(str)` Légendes des axes

`legend()` Afficher la légende

`xlim() / ylim()` Limites des axes

`figure(figsize=(w,h))` Taille du graphique

`savefig('nom.png')` Sauvegarder en image

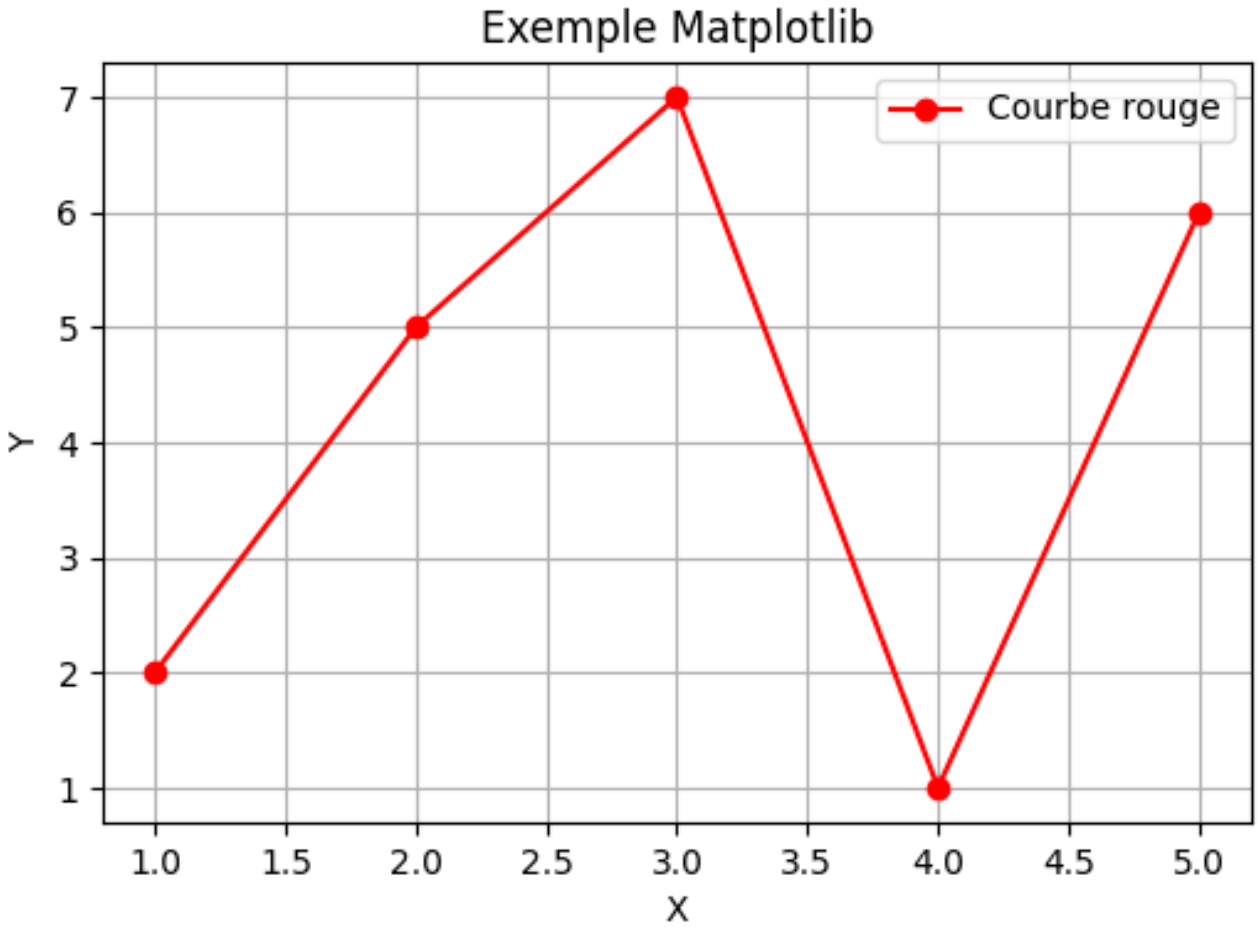
Prenons un exemple plus complet et affichons à nouveau le résultat, voici l'exemple de code :

```
mport matplotlib.pyplot as plt

# Données
x = [1, 2, 3, 4, 5]
y = [2, 5, 7, 1, 6]

# Création figure
plt.figure(figsize=(6,4))
plt.plot(x, y, color='red', marker='o', label='Courbe rouge')

plt.title("Exemple Matplotlib")
plt.xlabel("X")
```


Tous droits réservés à STUDI - Reproduction interdite

```
plt.ylabel("Y")
plt.legend()
plt.grid(e)
plt.show()
```

Exemple Matplotlib



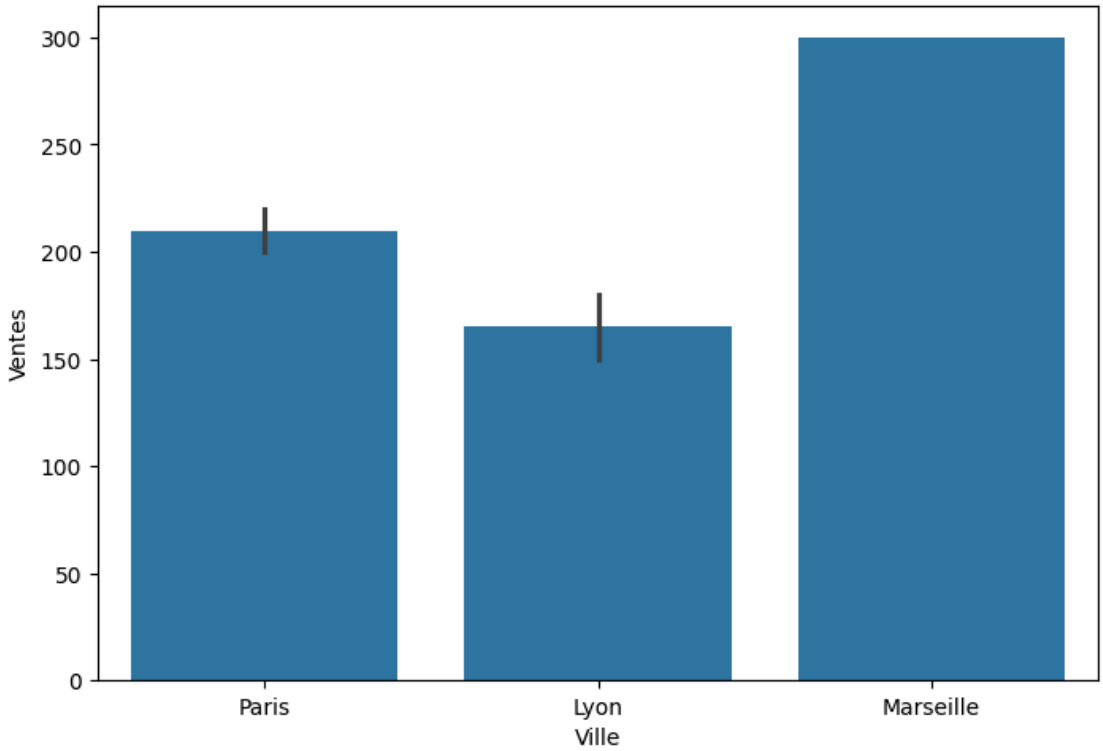
Passons à Seaborn, qui est une bibliothèque Python basée sur Matplotlib. Elle simplifie la syntaxe, ajoute des thèmes esthétiques et intègre Pandas pour ainsi fournir des graphiques plus avancés.

Voici un exemple simple :

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Jeu de données
df = pd.DataFrame({
    'Ville': ['Paris', 'Lyon', 'Marseille', 'Paris', 'Lyon'],
    'Ventes': [200, 150, 300, 220, 180]
})

sns.barplot(x='Ville', y='Ventes', data=df)

plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Exemple Seaborn



Important

Ici, Seaborn n'affiche que 3 barres, alors que nous avons « *5 villes* ». Il faut savoir que `sns.barplot()` ne montre pas toutes les valeurs individuelles, il regroupe les données par catégorie, ici la colonne « *Ville* », puis il affiche la moyenne des valeurs de chaque catégorie. Par défaut, `sns.barplot()` calcule la moyenne, ce qui donne :

- Paris → (200 + 220) / 2 = 210
- Lyon → (150 + 180) / 2 = 165
- Marseille → 300 (une seule valeur)

Si nous souhaitons voir chaque valeur, il nous faudra utiliser `sns.stripplot()` ou `sns.swarmplot()`.

Voici quelques méthodes importantes :

`barplot()` Diagramme en barres (moyennes + erreurs)

`countplot()` Comptage d'occurrences

`lineplot()` Courbe

`scatterplot()` Nuage de points

`histplot()` Histogramme

`boxplot()` Boîte à moustaches

`violinplot()` Distribution et densité

`heatmap()` Carte de chaleur

`pairplot()` Matrice de graphiques entre variables

`set_style()` Thème graphique

`set_palette()` Palette de couleurs

Une erreur récurrente est la petite subtilité suivante : nous n'avons pas besoin de `plt.plot()` ou de `plt.show()` si nous travaillons dans un Notebook Jupyter. Mais il faudra absolument les avoir dans un script !

Tous droits réservés à STUDI - Reproduction interdite

À travers les exemples que nous avons observés, les 2 librairies peuvent sembler très similaires, mais plus vous approfondirez les concepts, plus vous verrez leurs utilités distinctes. Matplotlib est idéale pour les rapports scientifiques et les graphiques précis. Seaborn est de très grande utilité dans l'analyse exploratoire et la data science rapide.

Scikit-learn

Scikit-learn est une librairie très populaire dans l'apprentissage automatique (*Machine Learning*). Elle fournit une collection complète d'algorithmes pour la classification, la régression, le regroupement (*clustering*), la réduction de dimensionnalité. La librairie Scikit-learn est conçue pour être simple d'utilisation, et est très bien documentée.

Le workflow standard de Scikit-learn pour construire un modèle de Machine Learning suit quelques étapes :

1. **Préparation des données** : les données doivent être sous forme numérique et nettoyées, pas de valeurs manquantes. Elles sont généralement divisées en un ensemble d'entraînement (pour apprendre) et un ensemble de test (pour évaluer le modèle).
2. **Le choix du modèle** : nous sélectionnons l'algorithme de Machine Learning adapté à notre problème (régression, classification, clustering).
3. **Entraînement du modèle** : le modèle apprend à partir des données d'entraînement.
4. **Prédiction** : nous utilisons le modèle entraîné pour faire des prédications sur de nouvelles données.
5. **Évaluation du modèle** : nous mesurons la performance du modèle sur l'ensemble de test pour voir s'il généralise bien.

Tous droits réservés à STUDI - Reproduction interdite

III Cycle de vie d'un projet de Machine Learning

Pour préparer au mieux un projet de Machine Learning, il convient d'avoir une approche méthodique. Un projet de Machine Learning en entreprise est bien plus qu'une exécution d'algorithmes. C'est un **processus structuré et encadré** qui exige une méthodologie rigoureuse pour aboutir à un modèle pertinent et performant. Penser à son cycle de vie garantit que chaque étape est menée de manière réfléchie, en passant de la collecte des données au déploiement du modèle.

La première phase est la **collecte de donnée :**, tout peut être considéré comme une donnée, ce qui peut rendre les choses plus complexes. Il s'agit en réalité ici de réunir les informations provenant parfois de sources variées (bases de données, API, fichiers Excel, formulaires, etc.) et de s'assurer de leur pertinence pour le **problème à résoudre**. Oui, il faut avant tout penser à résoudre un problème avant de collecter de la donnée à tout va.

Une fois que nous avons un guide de problématique à résoudre, ou d'amélioration suggérée après collecte de données dans les divers processus de l'entreprise, l'**analyse exploratoire** entre en jeu. C'est une étape pour comprendre la structure des données, identifier les corrélations entre les variables et repérer d'éventuels déséquilibres entre les classes (par exemple, dans un problème de détection de fraude, il y a beaucoup moins de transactions frauduleuses que de transactions normales). Ainsi, il faudra veiller à détecter les anomalies qui pourraient fausser notre analyse.

L'**évaluation du modèle** est une étape clé. Elle ne se limite pas à mesurer sa précision, mais va reposer sur des métriques adaptées au problème (précision, rappel, score). Il est essentiel de comprendre les résultats obtenus et d'interpréter les prédictions du modèle pour s'assurer que nous répondons aux besoins métiers. L'étape du **monitoring continu** du modèle en production est indispensable, car les données évoluent avec le temps, et un modèle qui est performant aujourd'hui ne le sera pas forcément demain. Et vice versa. C'est pour cette raison que le monitoring permet d'anticiper ces pertes de pertinence et de planifier les mises à jour nécessaires.

Une méthodologie qui a largement fait ses preuves est CRISP-DM (*Cross Industry Standard Process for Data Mining*). Elle permet de structurer le projet en 6 phases :

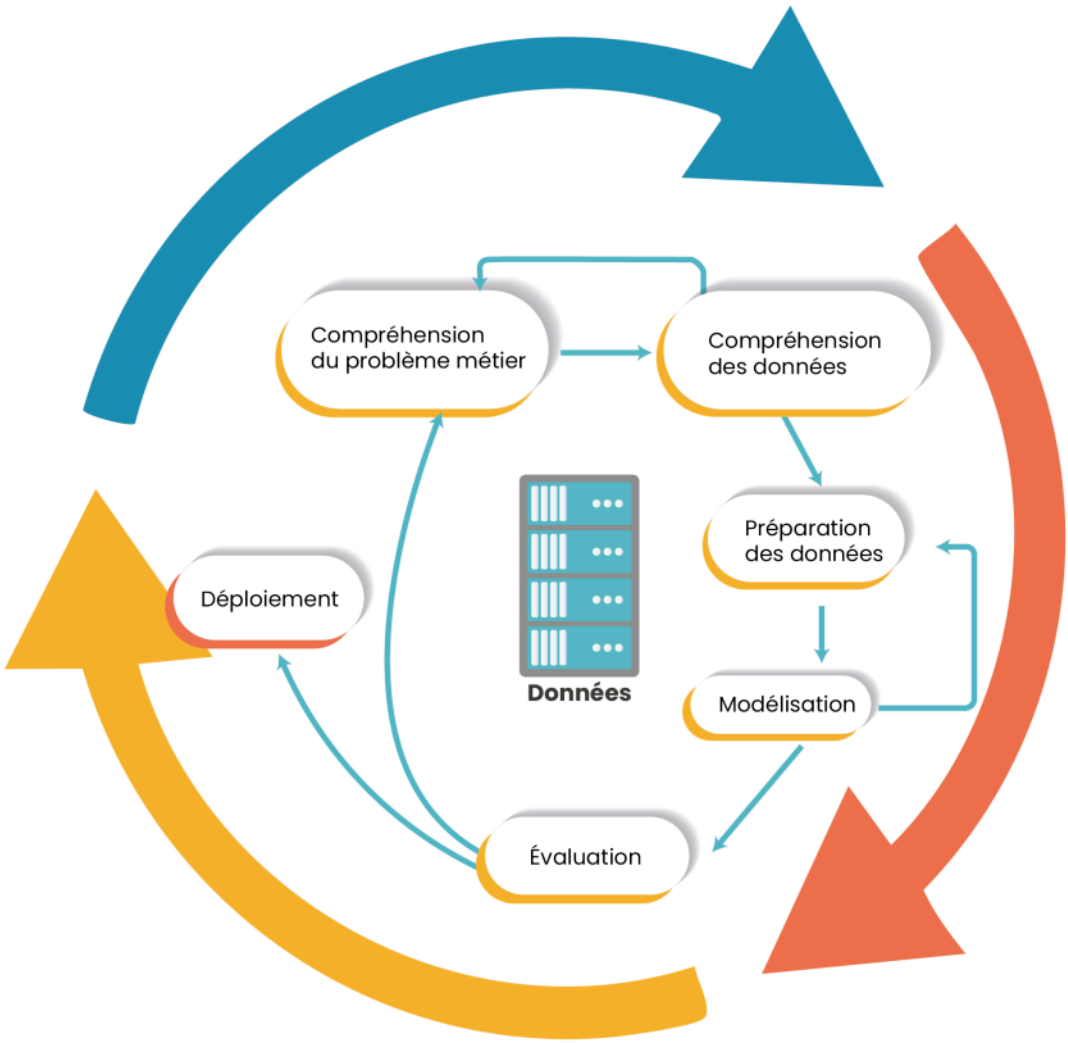
1. Compréhension du métier
2. Compréhension des données
3. Préparation des données
4. Modélisation
5. Evaluation
6. Déploiement

Cette méthodologie favorise une collaboration étroite entre les experts techniques (data scientists, data engineers) et les décideurs métier. Elle rend les projets plus fluides et mieux alignés sur les objectifs de l'entreprise.

Cette approche structurée et méthodique permet de passer d'une simple expérimentation à une solution d'intelligence artificielle robuste et créatrice de valeur.

Tous droits réservés à STUDI - Reproduction interdite

La méthode CRISP-DM



Tous droits réservés à STUDI - Reproduction interdite

IV Préparation et nettoyage des données

La préparation des données (et en fonction de certaines entreprises, nous pouvons aussi parler d'**export de données** pour traitement spécifique) est une étape très importante.

Imaginez que vous souhaitiez construire une maison. Vous avez les plans de l'architecte (votre objectif de machine learning) ainsi quel les outils, les ouvriers et les matériaux (les algorithmes et librairies). Avant de commencer, il faut s'assurer que le terrain est constructible, propre et préparé. En data science, ce terrain, ce sont vos données.

Parfois, la préparation du terrain peut prendre autant de temps que la construction en elle-même. **Quelles sont les données essentielles pour notre projet ?** Sont-elles toutes présentes ? **Sont-elles complètes ?** Des surprises peuvent apparaître ; des cellules, lignes, colonnes vides, des doublons, des formatages incorrects, notamment sur les dates. Des valeurs tellement extrêmes qu'elles peuvent sembler fausses, incorrectes. Tous ces problèmes dans nos données, s'ils ne sont pas traités, peuvent rendre notre modèle bancal, et ses prédictions peu fiables.

Le nettoyage consiste à « *balayer* » et à « *ranger* ». Nous décidons quoi faire au sujet des informations manquantes : nous pouvons les ignorer, les remplacer par une valeur moyenne, ou même essayer de les deviner. Généralement, concernant les doublons, ils sont souvent éliminés pour ne pas compter 2 fois la même chose. Nous pouvons corriger certaines fautes de frappe, mais avant tout, il faut s'assurer que les données sont dans le bon format. Et pour les valeurs extrêmes, les analyser et regarder si elles sont de vraies exceptions ou de simples erreurs à corriger.

Une étape importante est aussi la **mise en forme** de nos données pour qu'elles soient compréhensibles par les algorithmes (elles doivent par exemple être transformées en chiffres). Si nous avons des catégories comme « *petit* », « *moyen* », « *grand* », il faut transformer ces valeurs/étiquettes en chiffres. Si certaines valeurs sont très grandes et d'autres très petites, il faut les ajuster pour qu'elles soient sur la même échelle.

Maîtriser l'art du « *nettoyage* » et de la préparation des données est une compétence clé pour tout data scientist, et peut être fait à travers certains langages comme le SQL ; à cette même étape, bon nombre de requêtes peuvent être faites pour simplifier toute la suite du processus de nettoyage. C'est ce qui permet de construire une base solide pour le projet, et ainsi d'éviter les mauvaises surprises et de s'assurer que le modèle de Machine Learning, une fois entraîné, sera robuste et précis.

Tous droits réservés à STUDI - Reproduction interdite

V Structuration d'un projet de Machine Learning

La structuration d'un projet de Machine Learning se fait en bâtissant des fondations solides. Cette étape, souvent sous-estimée, reste essentielle pour garantir sa maintenabilité et sa reproductibilité, ainsi que la collaboration entre les différents développeurs. Beaucoup n'y prêtent que peu d'attention, ce qui est une grave erreur, car une architecture bien pensée est aussi la clé de sa durabilité et de sa capacité à évoluer.

En fonction des compétences de l'équipe, les projets de Machine Learning peuvent prendre différentes formes, allant du simple notebook au script Python plus complexe, voire parfois des applications web faisant appel à divers services. Sans dénigrer un outil, ou une méthodologie, les **notebooks** sont excellents pour l'analyse exploratoire des données, la visualisation rapide ou les démonstrations pédagogiques et commerciales. Ils permettent d'entrer dans le vif du sujet et de faire des itérations rapides et interactives. Cependant, pour des projet plus robustes et automatisés, il est bon de savoir passer d'un notebook à un **script bien structuré et versionné**. Un script permet une exécution plus stable et rapide en fonction des machines, et surtout une meilleure intégration dans des systèmes de production et de gestion des dépendances.

La **gestion des versions** est tout aussi indispensable pour garder une trace de toutes les évolutions du projet. Utiliser Git permet de revenir à des versions antérieures si nécessaire et de faciliter le travail en équipe. En Machine Learning, le versionning ne concerne pas uniquement le code, il est tout aussi important de versionner les modèles entraînés, les hyperparamètres utilisés pour leur entraînement, et les jeux de données sur lesquels ils ont été basés. Différents outils comme MLflow ou DVC existent pour gérer ces aspects, permettant ainsi la reproductibilité des expériences. Une bonne **organisation des fichiers** est aussi importante dans la gestion du projet : un dossier pour les données brutes, un pour les données traitées, un pour les modèles, etc. Cela permet de documenter le projet et de le rendre compréhensible de tous.

Structurer un projet de Machine Learning, c'est investir dans sa **pérennité**. En effet, cela permet de garantir la compréhension du projet par tous sur le long terme, à travers le travail et le nettoyage effectués, hier et aujourd'hui, afin que tout soit compris, modifié, réutilisé demain. Cela transforme une expérimentation en un actif durable pour l'entreprise.

Tous droits réservés à STUDI - Reproduction interdite

VI L'essentiel

Nous avons pu découvrir quelques librairies ; n'hésitez pas à explorer des sets de données et à essayer de les mettre en forme afin de faire quelques analyses exploratoires. Essayez de faire « *parler* » la donnée.

Jouez avec les exemples et essayez de produire des représentations graphiques de données que vous pourriez avoir à disposition.

Lisez la documentation de chaque librairie, car cela vous permettra de gagner du temps dans le futur et s'avèrera être un investissement rentable pour la suite. De plus, cela fait partie des bonnes pratiques à mettre en place pour un développeur et un data scientist.

Tous droits réservés à STUDI - Reproduction interdite

VII Synthèse

A. Flashcards

Algèbre linéaire

Manipulation de vecteurs et matrices pour organiser et traiter les données.

Gradient et descente de gradient

Méthode pour optimiser un modèle en suivant la pente de la fonction d'erreur.

Probabilités

Outils pour gérer l'incertitude et prédire la survenue d'événements.

Statistiques descriptives

Résumé des données par moyenne, médiane, mode, variance et écart-type.

NumPy

Librairie Python pour calculs numériques et manipulation de tableaux multidimensionnels.

Pandas

Librairie Python pour manipulation et analyse de données tabulaires.

Matplotlib & Seaborn

Librairies Python pour créer des visualisations graphiques des données.

Scikit-learn

Librairie Python pour modélisation, apprentissage et évaluation en Machine Learning.

CRISP-DM

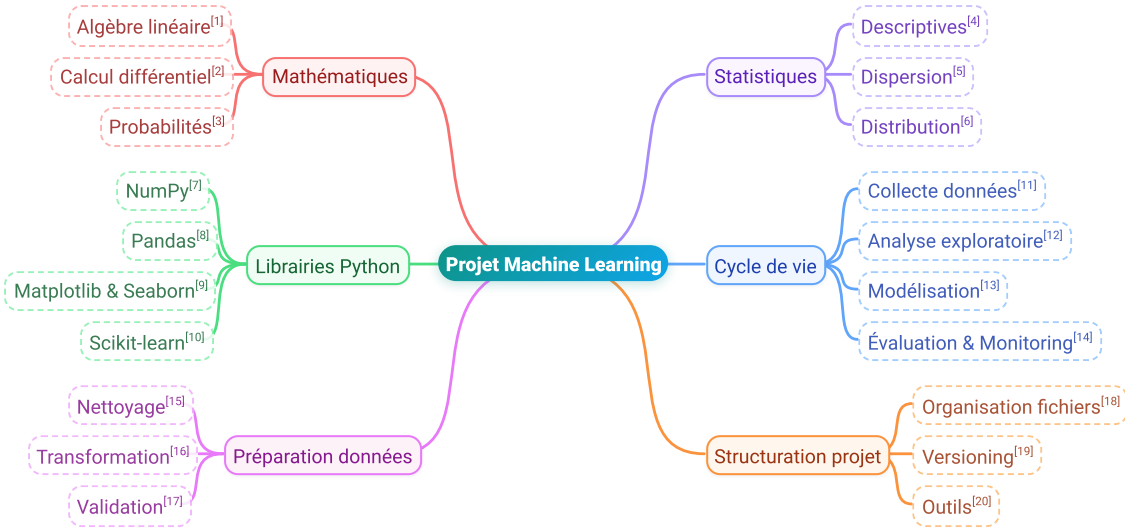
Méthodologie en 6 phases pour structurer un projet de data mining et Machine Learning.

La théorie en Bref

Fondamental

Un projet de **Machine Learning** repose sur des **fondamentaux mathématiques** comme l'**algèbre linéaire** (vecteurs, matrices), le **calcul différentiel** (dérivées, gradients) et les **probabilités** (incertitude, théorème de Bayes). Les **statistiques descriptives** permettent de résumer et de comprendre les données *via* la moyenne, médiane, mode, variance et écart-type. Les **librairies Python** essentielles incluent NumPy pour le calcul scientifique, Pandas pour la manipulation de données, Matplotlib et Seaborn pour la visualisation, et Scikit-learn pour le Machine Learning. La méthodologie CRISP-DM structure le projet en 6 phases clés, garantissant rigueur et collaboration. La préparation et le nettoyage des données sont essentiels pour assurer la qualité et la robustesse du modèle final.

Tous droits réservés à STUDI - Reproduction interdite



1. Vecteurs, matrices, multiplications
2. Dérivées, gradients, descente de gradient
3. Gestion incertitude, théorème de Bayes
4. Moyenne, médiane, mode
5. Variance, écart-type, étendue
6. Loi normale, variables aléatoires
7. Tableaux multidimensionnels, calcul vectorisé
8. DataFrames, manipulation données
9. Visualisation graphique
10. Modélisation, évaluation
11. Sources variées, pertinence
12. Structure, corrélations, anomalies
13. Choix modèle, entraînement
14. Métriques, suivi performance
15. Valeurs manquantes, doublons
16. Format, échelle, codage
17. Qualité, robustesse
18. Données, modèles, scripts
19. Code, modèles, données
20. Git, MLflow, DVC

Projet Machine Learning

Les conseils pro

Conseil

Comment préparer efficacement les données pour un projet de Machine Learning ?

- Identifier et traiter les valeurs manquantes (remplacement ou suppression).
- Supprimer les doublons pour éviter les biais.
- Convertir les données catégorielles en format numérique.
- Normaliser ou standardiser les données pour homogénéiser l'échelle.

Tous droits réservés à STUDI - Reproduction interdite

Comment choisir les librairies Python adaptées à chaque étape ?

- Utiliser NumPy pour les calculs numériques et manipulations matricielles.
- Employer Pandas pour le nettoyage et la manipulation des données.
- Recourir à Matplotlib/Seaborn pour visualiser et explorer les données.
- Appliquer Scikit-learn pour la modélisation et l'évaluation.

Comment structurer un projet de Machine Learning pour assurer sa maintenabilité ?

- Organiser les fichiers par type (données brutes, traitées, modèles).
- Utiliser un système de versioning (Git) pour le code et les modèles.
- Documenter chaque étape et les transformations des données.
- Adopter des scripts modulaires plutôt que des notebooks pour la production.

Comment appliquer la méthodologie CRISP-DM dans un projet ?

- Commencer par comprendre les besoins métier avant de collecter les données.
- Analyser et préparer soigneusement les données avant modélisation.
- Choisir et entraîner le modèle adapté au problème.
- Évaluer, déployer et monitorer le modèle en production régulièrement.

Tous droits réservés à STUDI - Reproduction interdite

VIII Auto-évaluation

A. Test

Exercice 1 : Quiz

[solution n°1 p. 32]

Question 1

Quel est l'intérêt principal d'utiliser une approche itérative pour développer un composant d'IA ?

- Réduire le nombre de lignes de code
- Livrer régulièrement des incréments fonctionnels et intégrer du feedback
- Éviter les tests unitaires
- Remplacer les data scientists par des développeurs classiques

Question 2

Dans le cycle CRISP-DM adapté à l'IA, quelle étape précède la modélisation ?

- Déploiement
- Évaluation
- Préparation des données
- Compréhension du métier

Question 3

Parmi ces choix, lequel est un exemple de « *dette de données* » ?

- Code spaghetti difficile à maintenir
- Modèle non documenté
- Données manquantes ou biaisées influençant l'apprentissage
- Absence d'API REST

Question 4

Quelle commande Python permet de connaître le nombre de dimensions d'un tableau NumPy arr ?

Tous droits réservés à STUDI - Reproduction interdite

`arr.dim()`

`arr.ndim`

`shape(arr)`

`len(arr)`

Question 5

Quelle commande Matplotlib est nécessaire pour afficher un graphique dans un script Python ?

`plt.render()`

`plt.plot()`

`plt.display()`

`plt.show()`

Question 6

Quelle méthode Matplotlib permet de tracer un nuage de points ?

`scatter()`

`bar()`

`plot()`

`hist()`

Question 7

Dans `plt.figure(figsize=(6,4))`, que représente `figsize` ?

La résolution en DPI du graphique

La taille de police des légendes

Les dimensions de la figure en pouces

La largeur des lignes

Tous droits réservés à STUDI - Reproduction interdite

Question 8

Quelle méthode Pandas permet de supprimer les lignes contenant des valeurs manquantes ?

- `df.remove()`
- `df.dropna()`
- `df.delete()`
- `df.clean()`

Question 9

Lorsque certaines valeurs extrêmes semblent aberrantes, que faut-il faire en premier ?

- Les supprimer systématiquement
- Les remplacer par zéro
- Les analyser pour déterminer si elles sont réelles ou erronées
- Les ignorer

Question 10

Dans la méthodologie CRISP-DM, quelle est la première étape ?

- Compréhension des données
- Déploiement
- Compréhension du métier
- Modélisation

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice

Analyse et visualisation des ventes

Voici un fichier csv :

(cf. ventes.csv)

Ville	Mois	Ventes
Paris	Janvier	200
Lyon	Janvier	150
Marseille	Janvier	300
Paris	Février	220
Lyon	Février	180
Marseille	Février	280

Question

[solution n°2 p. 35]

1. Importer le fichier CSV (« *ventes.cs* ») avec Pandas.
2. Afficher les 5 premières lignes du DataFrame.
3. Calculer le total des ventes par ville.
4. Tracer un graphique en barres avec Matplotlib représentant le total des ventes par ville.
5. Ajouter : un titre au graphique / Un label pour l'axe X (« *Ville* ») / Un label pour l'axe Y (« *Ventes totales* »)

Code de base à compléter :

```
#IMPORT


# 1. Importer les données


# 2. Afficher les premières lignes


# 3. Calculer le total des ventes par ville


# 4. Tracer un graphique en barres


# 5. Ajouter les titres et labels




# Afficher le graphique
```

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 28]

Question 1

Quel est l'intérêt principal d'utiliser une approche itérative pour développer un composant d'IA ?

- Réduire le nombre de lignes de code
- ✓ Livrer régulièrement des incréments fonctionnels et intégrer du feedback
- Éviter les tests unitaires
- Remplacer les data scientists par des développeurs classiques

L'approche itérative permet de découper le projet en petites étapes livrables. Chaque itération produit un composant fonctionnel, testé et amélioré grâce au retour des utilisateurs ou des parties prenantes ou techniques.

Question 2

Dans le cycle CRISP-DM adapté à l'IA, quelle étape précède la modélisation ?

- Déploiement
- Évaluation
- ✓ Préparation des données
- Compréhension du métier

Le cycle CRISP-DM commence par la compréhension du métier, puis la préparation des données, et ensuite vient la modélisation.

Question 3

Parmi ces choix, lequel est un exemple de « *dette de données* » ?

- Code spaghetti difficile à maintenir
- Modèle non documenté

Tous droits réservés à STUDI - Reproduction interdite

✓ Données manquantes ou biaisées influençant l'apprentissage

Absence d'API REST

La dette de données correspond aux problèmes liés à la qualité et à la représentativité, l'évolution des données qui dégradent les performances des modèles, c'est l'équivalent, pour les données de la dette technique dans le code.

Question 4

Quelle commande Python permet de connaître le nombre de dimensions d'un tableau NumPy arr ?

`arr.dim()`

✓ `arr.ndim`

`shape(arr)`

`len(arr)`

L'attribut `.ndim` d'un objet NumPy retourne le nombre de dimensions, par exemple, 1D,2D,3D. `shape` donne les tailles par dimension et non leur nombre.

Question 5

Quelle commande Matplotlib est nécessaire pour afficher un graphique dans un script Python ?

`plt.render()`

`plt.plot()`

`plt.display()`

✓ `plt.show()`

La commande `plt.show()` est indispensable pour afficher la figure à l'écran dans un script Python classique. Sans cet appel, le graphique reste en mémoire et ne s'affiche pas.

Question 6

Quelle méthode Matplotlib permet de tracer un nuage de points ?

Tous droits réservés à STUDI - Reproduction interdite

✓ `scatter()`

`bar()`

`plot()`

`hist()`

La méthode `plt.scatter(x, y)` trace des points individuels dans un plan, ce qui permet de visualiser les relations entre 2 variables continues.

Question 7

Dans `plt.figure(figsize=(6,4))`, que représente `figsize` ?

La résolution en DPI du graphique

La taille de police des légendes

✓ Les dimensions de la figure en pouces

La largeur des lignes

`figsize` définit la largeur et la hauteur de la figure en pouces. Cela permet d'ajuster la taille globale du graphique.

Question 8

Quelle méthode Pandas permet de supprimer les lignes contenant des valeurs manquantes ?

`df.remove()`

✓ `df.dropna()`

`df.delete()`

`df.clean()`

La méthode `dropna()` permet de supprimer les lignes ou colonnes selon les paramètres contenant des valeurs manquantes.

Tous droits réservés à STUDI - Reproduction interdite

Question 9

Lorsque certaines valeurs extrêmes semblent aberrantes, que faut-il faire en premier ?

- Les supprimer systématiquement
- Les remplacer par zéro
- ✓ Les analyser pour déterminer si elles sont réelles ou erronées
- Les ignorer

Avant toute suppression ou correction, il faut d'abord comprendre l'origine des valeurs extrêmes. Certaines peuvent être des erreurs de mesures, d'autres des phénomènes rares mais réels.

Question 10

Dans la méthodologie CRISP-DM, quelle est la première étape ?

- Compréhension des données
- Déploiement
- ✓ Compréhension du métier
- Modélisation

La compréhension du métier est l'étape initiale de CRISP-DM. Elle permet de cadrer le problème et de définir les objectifs, condition nécessaire avant l'analyse et le traitement des données.

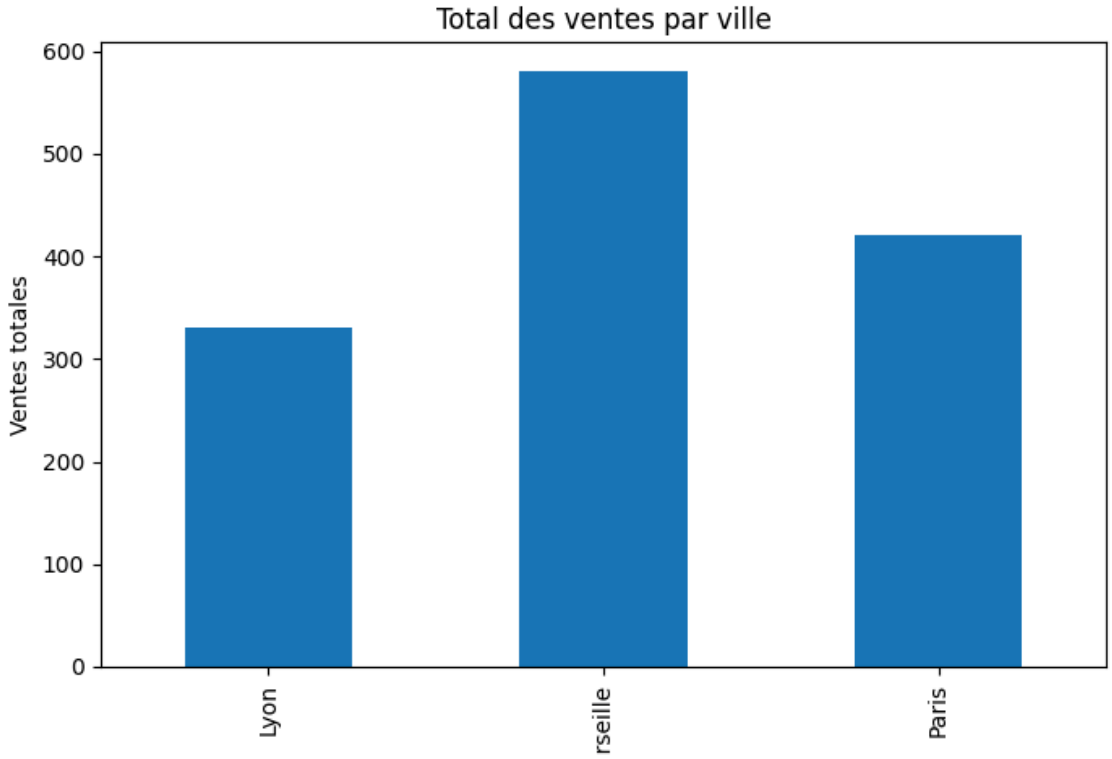
[exercice p. 31]

Solution n°2

```python
import pandas as pd
import matplotlib.pyplot as plt

# 1. Importer le fichier CSV
df = pd.read_csv("ventes.csv")

# 2. Afficher les 5 premières lignes du DataFrame
print(df.head())

# 3. Calculer le total des ventes par ville
ventes_par_ville = df.groupby("Ville")["Ventes"].sum()
print("\nTotal des ventes par ville :\n", ventes_par_ville)

```

Tous droits réservés à STUDI - Reproduction interdite

```
# 4. Tracer un graphique en barres
plt.figure(figsize=(8,5))
ventes_par_ville.plot(kind="bar")

# 5. Ajouter titre et labels
plt.title("Total des ventes par ville")
plt.xlabel("Ville")
plt.ylabel("Ventes totales")

# Afficher le graphique
plt.show()
```



Tous droits réservés à STUDI - Reproduction interdite